

Adamrit and Tally Sync Architecture

Adamrit and Tally Sync Architecture

Generated from the current Adamrit codebase on 2026-07-08.

Scope

This document covers two related systems:

1. The accounting module inside `adamrit.com`.
2. The sync and mirror architecture that connects Adamrit to TallyPrime.

The description below is based on the live source in:

- [src/components/accounting/AccountingPage.tsx](#)
- [src/components/accounting/TallyImportExport.tsx](#)
- [src/components/accounting/tally/TallyChrome.tsx](#)
- [src/lib/tally-auto-push.ts](#)
- [src/hooks/useAccountingData.ts](#)
- [src/components/tally/TallyPage.tsx](#)
- [src/components/tally/TallyVouchers.tsx](#)
- [src/pages/PatientLedger.tsx](#)
- [src/hooks/usePatientTransactions.ts](#)
- [src/hooks/usePaymentObligations.ts](#)

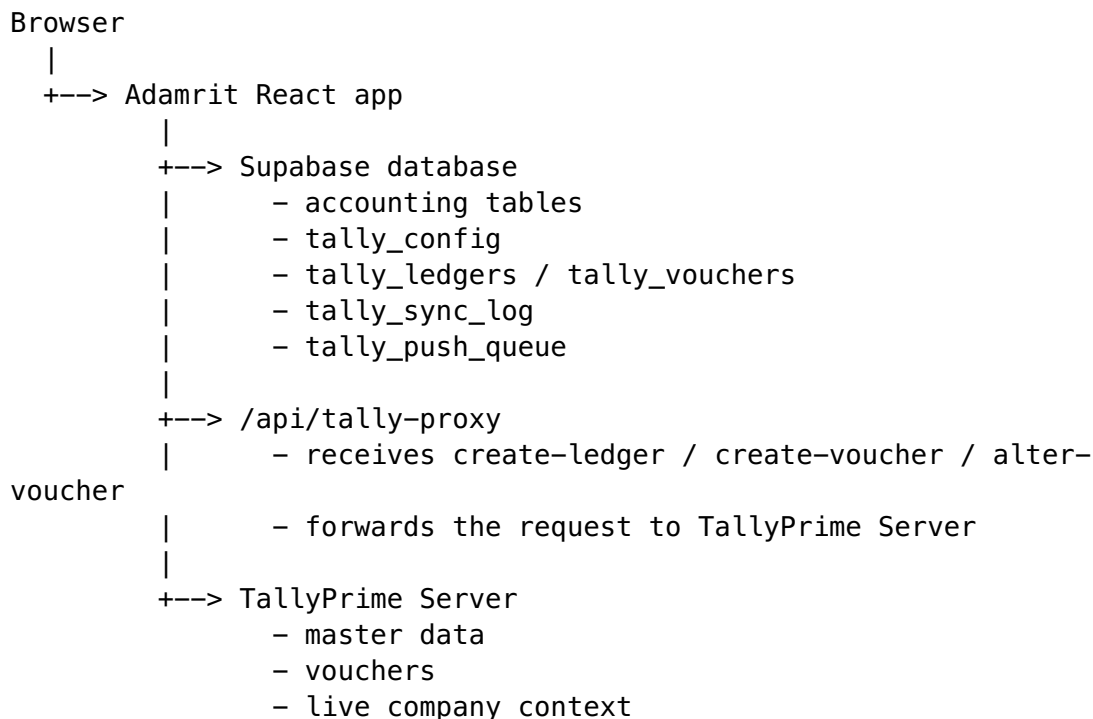
Executive Summary

Adamrit uses Supabase as the system of record and the browser as the primary application runtime. TallyPrime is an external accounting target that receives vouchers, ledgers, and selected business transactions through a push-oriented integration. The integration is not a single pipeline. It is a set of flows:

- interactive export/import from the accounting module,
- fire-and-forget auto-push for high-value workflows,
- a retry queue for failed pushes,
- a local mirror table for visibility inside Adamrit,
- and a separate Tally Live browser suite for inspection and reconciliation.

The accounting module is Tally-shaped on purpose. It exposes a Gateway-style shell, voucher entry, ledgers, day book, cash/book reports, reconciliation, and import/export screens. The Tally integration page sits alongside it and handles the actual XML exchange and the live sync surface.

High-Level Architecture



Adamrit is the source of truth for most operational data. Tally is the accounting target and also a validation surface. Some flows mirror completed Tally outcomes back into Supabase immediately so that the UI does not wait for a manual refresh.

Accounting Module

The accounting area is rendered from `AccountingPage`.

Shell and navigation

- `AccountingPage` owns the left navigation rail and the main content area.
- `TallyTopBar` renders the Tally-style header with the Go To search, menu row, print, help, and configuration behavior.
- The accounting module can jump between screens such as Gateway, Voucher Entry, Day Book, Ledger View, Trial Balance, Cash/Bank Book, Profit & Loss, Balance Sheet, Bills Outstanding, Bank Reconciliation, and Tally Live.

Main screens

The core screen map includes:

- `GatewayOfTally`

- Dashboard
- ChartOfAccounts
- AccountMasters
- VoucherEntry
- DayBook
- CashBankBook
- CashBankSummary
- LedgerView
- GroupSummary
- VoucherRegister
- ReceiptsPayments
- CostCentres
- EditLog
- ExceptionReports
- BillwiseOutstanding
- Banking
- TrialBalance
- BalanceSheet
- ProfitLoss
- CashFlow
- FundsFlow
- StatisticsScreen
- BankReconciliation
- BillsReceivable
- BillsPayable
- TallyImportExport
- TallyLive

This is important because the sync architecture is not isolated from the accounting module. The accounting module is where users create the records that later become Tally vouchers, and it is also where the imported Tally records are reviewed.

Data Model Used by Accounting

The accounting hooks fetch from the following major entities:

- chart_of_accounts
- patient_ledgers
- vouchers
- voucher_entries
- outstanding_invoices
- payment_transactions

The Tally integration layer adds a second set of entities:

- tally_config
- tally_ledger_mapping
- tally_ledgers
- tally_vouchers
- tally_sync_log
- tally_push_queue

Sync Architecture

1. Manual import/export

TallyImportExport is the explicit exchange screen.

It supports two separate paths:

- Tally XML masters import into Adamrit.
- Adamrit export of masters and vouchers into Tally XML.

Masters import

The import path accepts a Tally XML export, parses GROUP and LEDGER nodes, and inserts them into Adamrit:

- groups are deduplicated by name,
- parent groups are resolved in passes,
- ledgers are inserted in chunks,
- and existing names are skipped.

The parsed output is written into:

- ledger_groups
- ledgers

Masters export

The export path gathers:

- custom ledger_groups,
- active ledgers,

then emits a Tally XML envelope for import into Tally.

Voucher export

The voucher export path reads:

- posted vouchers from the accounting module,
- posted payment_vouchers,

then converts them to Tally voucher XML.

The export rules are:

- voucher category maps to Tally voucher type,
- debit lines become negative amounts,
- credit lines become positive amounts,
- payment vouchers treat the account ledger as the credit side,
- voucher numbers and narration are preserved.

2. Auto-push integration

`src/lib/tally-auto-push.ts` is the automation layer.

It handles the transactional push flows that should happen with minimal user friction:

- `pushLedgerToTally`
- `pushBillToTally`
- `pushPaymentToTally`
- `pushESICBillToTally`
- `pushInsuranceBillToTally`
- `pushInsurancePaymentToTally`
- `pushPharmacySaleToTally`
- `pushPaymentVoucherToTally`

Common behavior

Every push follows the same high-level shape:

1. Check whether Tally is active by reading `tally_config`.
2. Resolve the target company and server URL.
3. Resolve ledger mappings from `tally_ledger_mapping`.
4. Build a payload for `/api/tally-proxy`.
5. Send the payload with an action like `create-ledger`, `create-voucher`, `create-sales-voucher`, or `create-receipt-voucher`.
6. Log the result into `tally_sync_log`.
7. On success, mirror the result into local `tally_vouchers` where appropriate.
8. On failure, enqueue a retry job into `tally_push_queue`.

Mirror strategy

The mirror step is important.

When a push succeeds, the code inserts a matching row into `tally_vouchers`. This makes the local Tally views immediately visible without waiting for a later manual refresh from the external Tally server.

The mirrored row stores:

- voucher type,
- voucher number,
- date,
- party ledger,
- amount,
- narration,
- ledger entries,
- sync direction,
- sync status,
- Adamrit reference IDs,
- company ID,
- sync timestamp.

Retry strategy

Failed pushes are not dropped.

They are added to `tally_push_queue` with:

- push type,
- push action,
- serialized payload,
- reference ID,
- retry count,
- max retries,
- last error,
- next retry time,
- company ID.

This means the integration is eventually consistent, not fragile. A temporary Tally outage does not destroy the transaction; it delays completion.

3. Tally Live / inbound visibility

`src/components/tally/TallyPage.tsx` is the live Tally suite.

It exposes tabs for:

- Dashboard
- Ledgers
- Vouchers
- Cash Book
- Bank Book
- Reconciliation
- Stock Items
- Reports
- GST
- Bill Sync
- Mapping
- Create Voucher

The important part is the Vouchers tab:

- it reads `tally_vouchers`,
- pages 25 rows at a time,
- filters by date, voucher type, and sync status,
- and lets the operator inspect voucher details, edit, or cancel a voucher.

This is the local operational view of the Tally sync outcome.

Patient Ledger Flow

The patient ledger page is not itself a Tally screen, but it depends on accounting data and is part of the same financial picture.

PatientLedger combines:

- service transactions from `get_cash_book_transactions_direct`,
- advance payments from `advance_payment`.

The ledger then:

- filters by date range,
- sorts chronologically,
- computes debit, credit, and balance totals,
- and supports export to Excel.

That means the patient ledger is built on the same financial transaction model that later feeds the accounting and Tally flows.

Payment Obligations and Tally Mapping

`usePaymentObligations` shows how the accounting system carries Tally-specific references beyond simple vouchers.

It loads and manages:

- `payment_obligations`,
- `linked_tally_ledgers`,
- per-company `payment_obligation_ledgers`,
- ledger search by Tally company,
- and obligation CRUD operations.

This is useful because the daily payment allocation, expense planning, and other director workflows need stable Tally ledger links.

Source of Truth and Directionality

The integration is asymmetric:

- Adamrit is authoritative for hospital operations, billing, and patient activity.
- Tally is authoritative for accounting presentation and external accounting workflows.

Typical directions are:

- Adamrit -> Tally for bills, receipts, vouchers, and master data exports.
- Tally -> Adamrit for imported masters and live voucher visibility.

In practice, the system behaves as a local operational ledger with Tally as the external accounting mirror.

Failure Modes and Recovery

The codebase handles failure in layers:

- If Tally is not configured, the push is skipped or queued.
- If the HTTP push fails, the queue records the failure.
- If the Tally response is negative, the failure is logged and queued.
- If a mirror insert fails, the core push still succeeds because mirror writes are non-blocking.
- If import finds duplicates, they are skipped rather than aborted.

This is a pragmatic failure model. It favors data survival and later reconciliation over all-or-nothing transactions across systems that do not share a database.

Operational Notes

- Tally configuration is company-scoped through `tally_config`.
- Mapping is company-scoped through `tally_ledger_mapping`.
- The accounting module uses a Tally-style shell to reduce operator training cost.
- Voucher paging is important because the local Tally voucher table can grow large.
- The system currently relies on browser-side Supabase access and an `/api/tally-proxy` bridge, not a dedicated backend service layer.

Architecture Constraints

The current architecture has a few important constraints:

1. Synchronization is not a strict two-phase commit.
2. The app depends on Supabase RLS and the proxy endpoint for security boundaries.
3. A successful push can still be mirrored later, so local and external states may be briefly out of sync.
4. Some screens are Tally-style replicas rather than full accounting engines, so the visual and data models are intentionally opinionated.

Bottom Line

The Adamrit-Tally integration is a browser-driven accounting system with:

- a Tally-style accounting module,
- explicit XML import/export,
- automated push helpers for common transactions,
- company-specific ledger mapping,
- a local voucher mirror,
- and a retry queue for resilience.

That combination is enough to support live operations without forcing every accounting action through Tally in real time.

Source References

- [src/components/accounting/AccountingPage.tsx](#)
- [src/components/accounting/TallyImportExport.tsx](#)
- [src/components/accounting/tally/TallyChrome.tsx](#)
- [src/lib/tally-auto-push.ts](#)
- [src/hooks/useAccountingData.ts](#)
- [src/components/tally/TallyPage.tsx](#)
- [src/components/tally/TallyVouchers.tsx](#)
- [src/pages/PatientLedger.tsx](#)
- [src/hooks/usePatientTransactions.ts](#)
- [src/hooks/usePaymentObligations.ts](#)